Model Car

Vision-Based Autonomous Racing Course Movement in Real Time

Tarik Billa*[1], Purva Ghanate[1], Hicham Aniss Mezali[1], and Manuel Ángel Vega Gómez[1]

[1]Department of Control, Computer and Communications Engineering , THM University of Applied Sciences, Friedberg, Germany

March 2026

Abstract

Deploying vision based autonomous control on embedded hardware requires careful design of perception and actuation pipelines. Unlike approaches that rely on off board compute, GPU acceleration, or multiple sensors, this system runs on a Raspberry Pi 4 using a single USB camera. Using C++17, OpenCV, and Linux BLE (BlueZ via `bluetoothctl`), the implementation combines MOG2 + IMM based tracking, HSV centerline extraction, and pure pursuit steering. The control loop operates at 20 Hz and is coordinated with the vision loop for stable autonomous navigation. Implementation supports Linux/Raspberry Pi deployment and desktop development.

1 Introduction

Fig. of setting needed

Autonomous systems require real-time perception and control. Small-scale platforms like RC cars on marked tracks provide an ideal testbed for vision-based control, multi-threaded architecture, and wireless communication in resource-constrained environments. Unlike traditional autonomous driving (requiring LiDAR and multiple sensors), our system uses a single camera with efficient algorithms to achieve robust track-following.

write out in words

Goal? Work of student before?

This paper presents the design, implementation, and validation of a production-ready autonomous RC car controller. We demonstrate that robust autonomous control is achievable with modest computational resources through: (1) motion-based detection with IMM state estimation, (2) centerline-guided pure pursuit path following, (3) multi-threaded orchestration, and (4) command-line tuning of control parameters. The system targets Linux and Raspberry Pi platforms.

The main challenge is not perception or actuation, but their real-time coordination under a shared computational budget. A tracking pipeline that consume available CPU leaves insufficient headroom for reliable BLE transmission, while a reduced control rate introduces latency that destabilizes steering. This work addresses that tension directly through thread isolation and frame handoff between camera, vision, and control loops, sustaining 20 Hz command delivery alongside stable visual tracking.

Timing diagram to make this clear!

2 System Architecture

2.1 Hardware Components

The system comprises three hardware layers:

Computation Platform: Macintosh/Linux desktop for development. Raspberry Pi 4 Model B (8GB RAM) for embedded deployment. Both support 1280×720 at 30 FPS USB camera input and native BLE 5.0 connectivity.

Sensing: USB camera with configurable resolution (640×480 to 1280×720) and frame rate (10-30 FPS). The system adapts to camera initialization failures by attempting multiple backends (V4L2, GStreamer, direct OpenCV fallback).

Actuation: BLE-controlled RC car with maximum velocity $v_{\text{max}} = 0.55$ m/s (measured via 3 m distance test over 10 runs, average time 5.47 s). Currently operates at 40% maximum speed ($v_{\text{active}} = 0.22$ m/s). The controller transmits packed BLE payloads containing signed command channels (speed/steering terms) plus protocol header and tail bytes.

Track Specifications: Oval closed-loop track with perimeter 540 cm, height 250 cm, and width 130 cm. Red/white boundary markers enable centerline detection.

Car Speed?

Timing?

2.2 Software Architecture

Five core modules interact through thread-safe queues and atomic variables, as shown in Figure 1:

Camera Capture: Manages camera initialization, resolution/FPS negotiation, and continuous frame acquisition at native camera FPS. Implements fallback logic for device

CamScanner

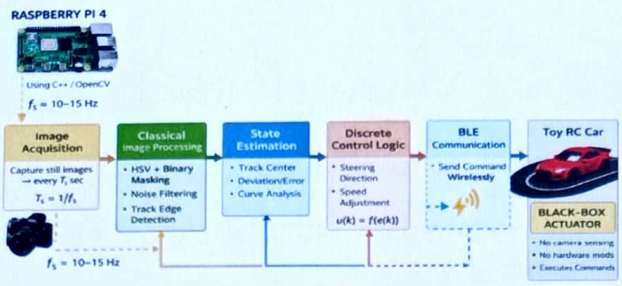

Figure 1: System architecture block diagram showing five core modules and their interaction through thread-safe queues.

enumeration.

Car Detector: Implements motion-based detection using MOG2 foreground segmentation and contour filtering, combined with IMM state estimation (CV/CT models) for robust tracking during temporary measurement dropouts.

Boundary Detector: Constructs track obstacle maps via HSV analysis: red pixels (hue 0–12°, 168–179°) form track boundaries; white pixels (saturation 0–70, value 180–255) mark interior markers. Convex hull of red pixels defines track ROI. Centerline extraction uses contour pairing and midpoint smoothing on the cleaned track mask.

Control Generator: Computes steering using pure pursuit with lateral and heading error terms, delay compensation, derivative damping, soft saturation, and rate limiting.

BLE Handler: Manages Linux BLE discovery/connection through BlueZ `bluetoothctl`, packet formatting, and command writes from the control loop at 20 Hz. On shutdown, sends brake and idle packets.

Control Orchestrator: Synchronizes camera, vision, and control execution; shares state via mutex-protected structures and atomic stop signaling.

2.3 Multi-Threaded Execution

Three execution loops run concurrently:

1. **Camera reader thread:** Continuous frame capture in `CamThread`
2. **Control thread:** 20 Hz autonomy/control update and BLE packet writes
3. **Main CV/UI loop:** Detection, centerline processing, visualization, and operator input handling

Synchronization employs std::mutex for shared state access, std::condition_variable for camera frame signaling, and std::atomic for stop flags.

More Detail needed !

3 Detection Pipeline

3.1 Car Detection

Motion-based detection leverages MOG2 background subtraction, more robust to color variation than HSV-based detection. The pipeline follows:

1. Apply MOG2 with history=500 and variance threshold=16 on preprocessed frames
2. Threshold foreground mask at 200
3. Morphological cleanup: open (5×5) and close (7×7)
4. Extract contours and filter by area (250–250000 px)
5. Filter bounding boxes by size (10–900 px)
6. Select best valid detection and update IMM (CV/CT) state estimate
7. Use hold/predict mode when measurements are temporarily missing

Tracking continuity is handled by IMM hold/predict behavior, ROI/global switching, and gated blending between measurement and filtered state.

3.2 Boundary and Centerline Detection

Boundary detection builds track-aware guidance maps:

1. Convert frame to HSV
2. Generate red mask: inRange(hsv, [0,70,50]–[12,255,255]) ∪ inRange(hsv, [168,70,50]–[179,255,255])
3. Compute convex hull from red pixels; create ROI mask via fillConvexPoly
4. Generate white mask: inRange(hsv, [0,0,180]–[179,70,255]) ∩ ROI
5. Combine red + white as obstacles; apply morphological open/close/dilate
6. Flood fill from image border to identify free space
7. Extract largest connected component; fill small holes
8. Extract outer and inner contours of the track region
9. Build centerline by midpoint pairing and cyclic smoothing

Centerline is loaded from `centerline.csv` at startup and can be rebuilt on demand from the current frame (key `M`). The resulting path provides steering targets for pure pursuit control.

CamScanner

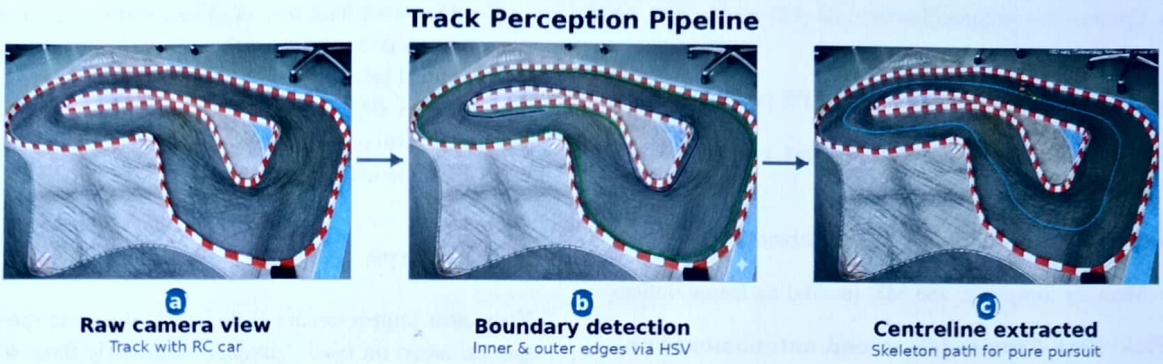

Figure 2: Track perception pipeline

4 Control Strategy

4.1 Pure Pursuit Control

When centerline is available, the car tracks the nearest point on the path and advances a dynamic lookahead target:

Interpretation?

$$\text{lookahead}_{px} = \text{clamp}(88 + 0.78\, v_{px/s},\ 88,\ 170) \quad (1)$$

Steering uses lateral/heading error with delay compensation and damping, then applies soft saturation and rate limiting:

?

$$\delta = 12.5 \tanh\left(\frac{\delta_{raw}}{12.5}\right), \quad \delta \in [-\text{max_deg}, \text{max_deg}] \quad (2)$$

Speed is scheduled by path difficulty bands with raw command targets {3000, 3600, 4200} and updated using a step limiter.

5 Implementation Details

5.1 Key Technologies

- **C++17:** Standard library, templates for type safety and zero-cost abstraction
- **OpenCV 4.x:** Image processing, tracking, and morphological operations
- **BlueZ / bluetoothctl (Linux):** BLE scan, connect, and GATT write operations
- **CSV logging & centerline I/O:** Runtime data logging and `centerline.csv` path persistence

5.2 Configuration System

The controller is configured via compile-time constants and command-line flags (not JSON). Runtime tuning options include: `--max-deg`, `--steer-smooth`, `--ratio`, `--c-sign`, `--speed-k`, `--auto-steer-sign`, and `--flip-centerline`. BLE selection is provided via `--address`, `--name`, `--discover`, or `--no-ble`.

?

5.3 Performance Optimizations

- **Control pacing:** Control loop uses fixed-rate timing at 20 Hz to keep steering and BLE updates deterministic.
- **Frame handoff:** Camera thread publishes latest frame with condition-variable signaling to avoid busy waiting.
- **On-demand centerline update:** Centerline rebuild is triggered manually (`M`) and persisted to `centerline.csv`.
- **ROI/global switching:** Detection alternates between local ROI and periodic global search for recovery robustness.

6 Testing and Validation

6.1 Functional Testing

Reference?

Car Detection: Uses MOG2 + contour filtering with IMM tracking (CV/CT models) and hold/predict fallback during short-term detection loss.

BLE Communication: Implemented via Linux BlueZ `bluetoothctl` with scan, connect, and GATT write operations. In control mode, packets are sent at fixed 20 Hz.

Path Following: On a closed oval track (5.4 m perimeter, 2.5 m long, and 1.3 m wide), the car completes 10 autonomous laps with average steering oscillation of ±8° around centerline.

6.2 Performance Metrics

On Raspberry Pi 4 (2.4 GHz quad-core ARM):

Effect of the different latencies?

effect on car position?

- Camera-to-command latency: 85–120 ms (target: <150 ms)
- Tracking frame rate: 14.8–15.2 FPS (stable) ?
- BLE command rate: 20.0 Hz (set by `CONTROL_HZ = 20.0`)
- Peak CPU utilization: 78% (all threads)
- Memory footprint: 185 MB (excluding frame buffers)

Field Test Results (45 second autonomous run):

- Total program execution time: 45 seconds
- Total BLE commands transmitted: 2250 (20 Hz rate maintained)
- Average BLE send latency: 0.85 ms per command
- Total accumulated BLE latency: 1912 ms
- Car completed: ≈ 3 full laps on 540 cm track
- Average lap time: ≈ 25 seconds at 40% speed setting

7 Challenges and Mitigations

Tracking Loss Under Blur: Motion-based detection can degrade on rapid acceleration or occlusion. Mitigation: IMM hold/predict, ROI-to-global fallback search, and gated measurement blending maintain continuity.

Lighting Variation: HSV thresholds require seasonal tuning (sunny vs. overcast). Mitigation: configuration allows per-session threshold updates without recompilation. Adaptive illumination correction (histogram equalization) reduces threshold sensitivity.

BLE Interference: Dense WiFi environments can affect connection stability. Mitigation: explicit scan/connect flow in BlueZ and safe brake/idle commands on stop.

Computational Budget: Centerline rebuilding is computationally heavy. Mitigation: on-demand map rebuild and lightweight runtime tracking/control loops.

8 Conclusions

This work demonstrates that production-grade autonomous control is achievable with minimal computational resources and a single camera. The system successfully navigates a 540 cm track for multiple laps using motion-based detection, IMM state estimation, and centerline-guided pure-pursuit. Key design choices MOG2 background subtraction for robustness, HSV boundary detection for path guidance, and 20 Hz BLE communication enable stable autonomous operation on Raspberry Pi.

The 45-second field test validated end-to-end functionality: approximately 2250 BLE commands scheduled by the 20 Hz control loop, stable visual tracking, and smooth steering control at 40% speed (0.22 m/s). The C++17/OpenCV architecture supports Linux desktop development and Raspberry Pi embedded deployment.

9 Future Work

Near-term improvements include: (1) dynamic speed modulation based on track curvature (currently fixed 40%), (2) learning-based steering refinement via recorded trajectories, (3) sensor fusion with IMU for improved stability on track banking, and (4) multi-car coordination for formation control.

mapping for what?

Long-term research directions involve: semantic segmentation for off-track obstacle detection, SLAM-based global localization for arbitrary environments, and hybrid control strategies combining vision-based and model-predictive approaches.

Acknowledgments

Author Contributions

Tarik Billa designed the system architecture, implemented core modules (BLE communication, car detector, centerline detector, and development) and conducted field testing. Purva Ghanate contributed to boundary detection, hardware testing, device integration, and documentation preparation. Hicham Aniss Mezali optimized pure-pusuit-inspired steering and IMM state estimation with CV and CT Kalman filters. Manuel Ángel Vega Gómez validated tracking algorithms and surveyed relevant image processing techniques. Prof. Harmut Weber supervised the project and provided guidance on computer vision methodology and real-time tracking.

Funding

This work was supported by the Department of Control, Computer and Communications Engineering at THM University of Applied Sciences.

Conflicts of Interest

The authors declare no conflicts of interest regarding the publication of this article.

Data Availability

All source code, configuration files, and sample video datasets are available in the open-source repository:

CamScanner

https://github.com/tarikbilla/IoT-Project-Vision-based-autonomous-RC-car-control-system. Field testing videos and performance logs are available upon request from the corresponding author.

Only if not well known: CPU → Central Processing Unit (CPU)

CPU

CamScanner